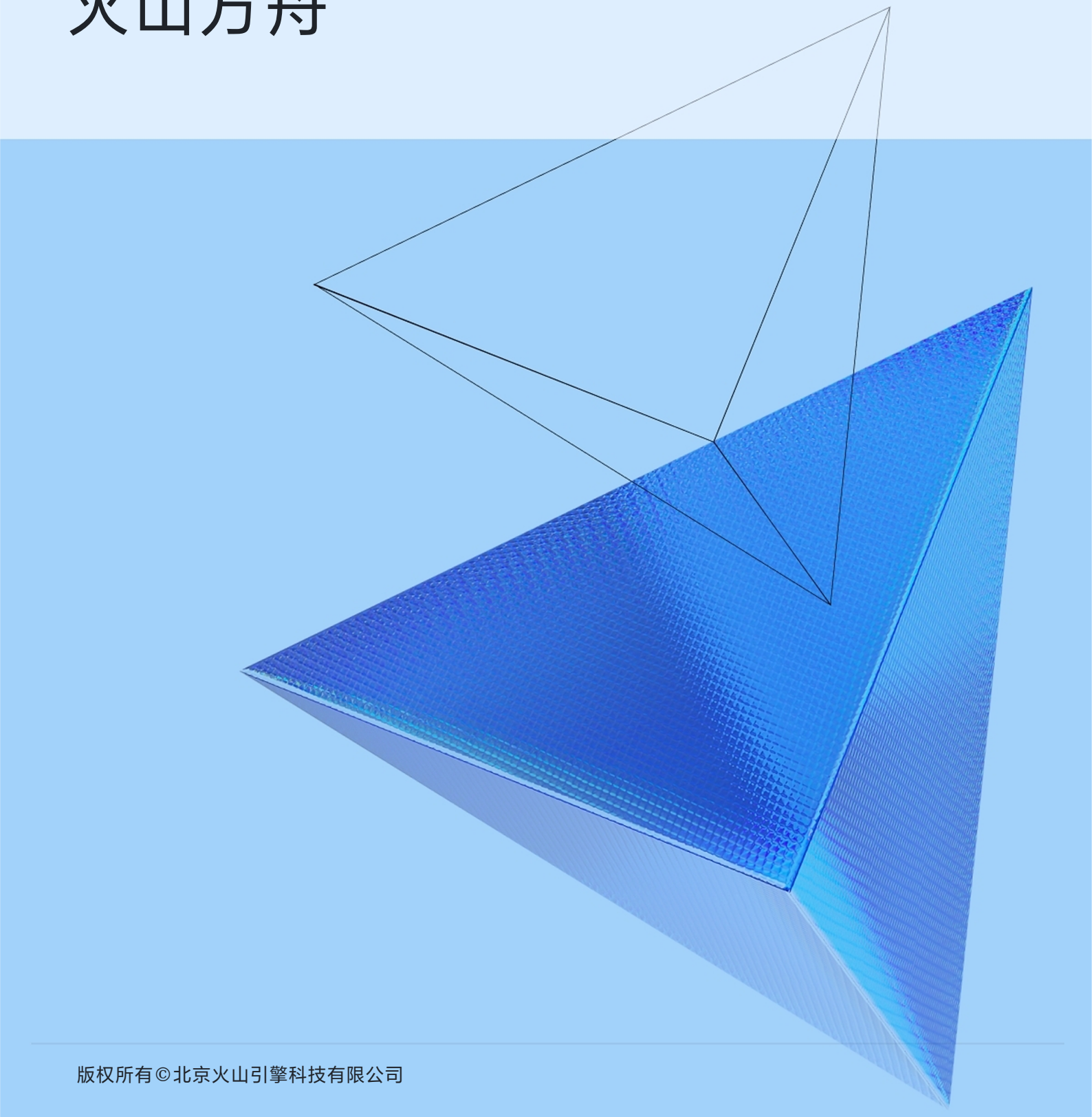
火山引擎

接入语音模型

火山方舟



版权所有©北京火山引擎科技有限公司

法律声明

本《火山方舟》的所有内容，包括但不限于文字、商标、架构、图示、图片、页面布局等，其知识产权（著作权、商标权、专利权、商业秘密等）归属于北京火山引擎科技有限公司及其关联公司（火山引擎），非经火山引擎书面同意，任何个人和组织不得复制、使用、修改、转发或以任何违反本《火山方舟》所承载的目的进行传播。

本《火山方舟》陈述内容仅作为产品的通用性介绍和参考性指引，火山引擎保留按“现状”和“当前可用”的形式提供产品和服务的权利。火山引擎不对本《火山方舟》中所载的产品功能、性质、质量、标准等内容进行明示或默示的保证和承诺，最终以您与火山引擎实际签署的协议为准。

如您发现本《火山方舟》有任何错误或歧义，或发现有对本《火山方舟》、产品本身的侵权行为，请与火山引擎取得联系。

联系方式：service@volcengine.com，400-850-0030
（周一至周五 10:00-18:00）

版权所有©北京火山引擎科技有限公司

目录

法律声明----------1
目录----------1
1. 接入语音模型----------1

版权所有©北京火山引擎科技有限公司

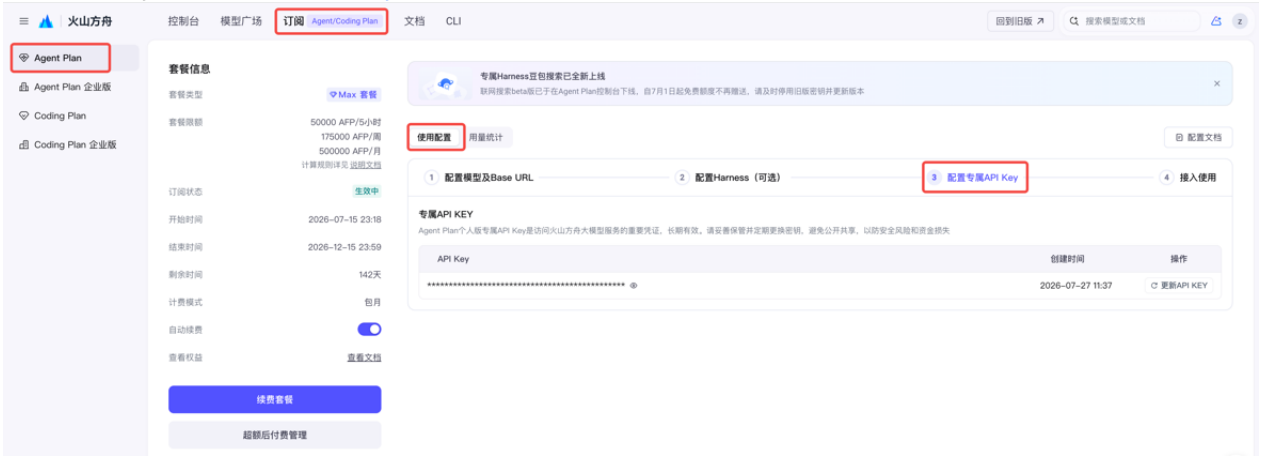
1. 接入语音模型

方舟 Agent Plan 提供的语音模型支持 **语音合成**（TTS）和 **语音识别**（ASR）两大能力，可以通过 WebSocket 和 HTTP 协议接入使用语音服务。

核心配置

在接入语音模型前，需要准备以下信息：

- **专属 API Key**： 获取专属 API Key



- **支持的模型**：
  - 豆包语音合成模型2.0（doubao-seed-tts-2.0）：接口中参数 `X-Api-Resource-Id` 需配置为 `seed-tts-2.0`
  - 豆包流式语音识别模型2.0（doubao-seed-asr-2.0）：接口中参数 `X-Api-Resource-Id` 需配置为 `volc.seedasr.sauc.duration`
- **接口信息**： 流式语音合成（Streaming TTS）、流式语音识别（ASR Streaming）

流式语音合成（Streaming TTS）

流式语音合成支持将文本实时转换为语音，适用于实时文本流输入、音频内容生成场景。

接口信息

根据具体场景选择合适的语音合成 API，对应的参数及细节可参见对应的 API 文档。

接口类型	接口协议	接口地址	推荐场景	API 文档
双流接口	WebSocket	`wss://openspeech.bytedance.com/api/v3/plan/tts/bidirection`	支持流式发送文本、流式接收音频，适合实时对话等低延迟场景。	WebSocket 双向流式文档
单流接口	WebSocket	`wss://openspeech.byteda`	一次性发送全部文本，流式接收	WebSocket 单向流式文档

版权所有©北京火山引擎科技有限公司

接口类型	接口协议	接口地址	推荐场景	API 文档
		nce.com/api/v3/plan/tts/unidirectional/stream	音频片段，适合长文本播报。	
HTTP 接口	HTTP POST	https://openspeech.bytedance.com/api/v3/plan/tts/unidirectional	一次性发送文本，一次性返回完整音频，适合简单场景或非实时调用。	HTTP Chunked 单向流式文档

代码示例

> **注意**
>
> - 代码示例中 `X-Api-Key` 需要设置为 专属 API Key。
> - 在运行 **双流接口** 、 **单流接口** 的代码示例前，请先创建 `protocols.py` 文件。该文件封装了与语音合成服务端进行 WebSocket 通信的协议与消息处理逻辑，代码如下：
>   【附件下载】: protocols.py，大小为

双流接口

```
import copy
import json
import logging
import uuid
import asyncio

import websockets

from protocols import (
    EventType,
    MsgType,
    finish_connection,
    finish_session,
    receive_message,
    start_connection,
    start_session,
    task_request,
    wait_for_event,
)

logging.basicConfig(level=logging.INFO)
```

```
logger = logging.getLogger(__name__)
logging.getLogger('protocols').setLevel(logging.WARNING)

URL = "wss://openspeech.bytedance.com/api/v3/plan/tts/bidirection"

async def main():
    # 要合成的文本
    text = "你好，欢迎使用语音合成服务。"

    # 连接服务器
    headers = {
        "X-Api-Key": "your_api_key",
        "X-Api-Resource-Id": "seed-tts-2.0",
        "X-Api-Connect-Id": str(uuid.uuid4()),
        "X-Control-Require-Usage-Tokens-Return": "*"
    }

    logger.info(f"Connecting to {URL}")
    websocket = await websockets.connect(
        URL, additional_headers=headers, max_size=10 * 1024 * 1024
    )
    logger.info(
        f"Connected to WebSocket server, Logid: {websocket.response.headers['x-tt-logid']}",
    )

    try:
        # 开始连接
        await start_connection(websocket)
        await wait_for_event(
            websocket, MsgType.FullServerResponse, EventType.ConnectionStarted
        )

        # 按句子分割处理
        sentences = text.split("。")
        audio_received = False

        for i, sentence in enumerate(sentences):
            if not sentence:
                continue

            # 每次会话可以有不同的参数
            base_request = {
                "req_params": {
                    "text": sentence,
                    "speaker": "zh_female_gaolengyujie_uranus_bigtts",
                    "audio_params": {
                        "format": "mp3",
```

```
            "sample_rate": 24000,
            "enable_timestamp": False,
        }
    },
}

# 开始会话
start_session_request = copy.deepcopy(base_request)
start_session_request["event"] = EventType.StartSession
session_id = str(uuid.uuid4())
await start_session(
    websocket, json.dumps(start_session_request).encode(), session_id
)
await wait_for_event(
    websocket, MsgType.FullServerResponse, EventType.SessionStarted
)

# 逐字发送
async def send_chars():
    for char in sentence:
        synthesis_request = copy.deepcopy(base_request)
        synthesis_request["event"] = EventType.TaskRequest
        synthesis_request["req_params"]["text"] = char
        await task_request(
            websocket, json.dumps(synthesis_request).encode(), session_id
        )
        await asyncio.sleep(0.005)  # 每个字符间隔5ms

    await finish_session(websocket, session_id)

# 在后台开始发送字符
send_task = asyncio.create_task(send_chars())

# 接收音频数据
audio_data = bytearray()
while True:
    msg = await receive_message(websocket)

    if msg.type == MsgType.FullServerResponse:
        if msg.event == EventType.SessionFinished:
            break
        if msg.payload:
            payload = json.loads(msg.payload)
            logger.info(f"Server response: {payload}")
    elif msg.type == MsgType.AudioOnlyServer:
        if not audio_received and len(audio_data) > 0:
            audio_received = True
```

```python
            audio_data.extend(msg.payload)
        else:
            raise RuntimeError(f"TTS conversion failed: {msg}")

    # 等待 send_chars 完成
    await send_task

    # 保存音频文件
    if audio_data:
        filename = f"bidirectional_session_{i}.mp3"
        with open(filename, "wb") as f:
            f.write(audio_data)
        logger.info(f"Audio received: {len(audio_data)}, saved to {filename}")

    if not audio_received:
        raise RuntimeError("No audio data received")

finally:
    # 结束连接
    await finish_connection(websocket)
    msg = await wait_for_event(
        websocket, MsgType.FullServerResponse, EventType.ConnectionFinished
    )
    if msg.payload:
        payload = json.loads(msg.payload)
        if "usage" in payload:
            logger.info(f"Usage: {payload['usage']}")
    await websocket.close()
    logger.info("Connection closed")

if __name__ == "__main__":
    asyncio.run(main())
```

单流接口

```python
import asyncio
import json
import logging
import uuid

import websockets

from protocols import EventType, MsgType, full_client_request, receive_message
```

```
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

async def main() -> None:
    headers = {
        "X-Api-Key": "your_api_key",
        "X-Api-Resource-Id": "seed-tts-2.0",
        "X-Control-Require-Usage-Tokens-Return": "*"
    }

    body = {
        "req_params": {
            "speaker": "zh_female_gaolengyujie_uranus_bigtts",
            "text": "这是一段较长的文本内容，单向流式会一次发送全部文本，然后流式返回音频片
段。",
            "audio_params": {
                "format": "mp3",
                "sample_rate": 24000,
            }
        }
    }

    websocket = await websockets.connect(
        "wss://openspeech.bytedance.com/api/v3/plan/tts/unidirectional/stream",
        additional_headers=headers,
        max_size=10 * 1024 * 1024,
    )
    logger.info(
        f"Connected to WebSocket server, Logid: {websocket.response.headers.get('x-tt-logid', '')}"
    )

    try:
        await full_client_request(websocket, json.dumps(body).encode())

        audio_data = bytearray()
        while True:
            msg = await receive_message(websocket)

            if msg.type == MsgType.FullServerResponse and msg.event == EventType.SessionFinished:
                break
            if msg.type == MsgType.AudioOnlyServer and msg.payload:
                audio_data.extend(msg.payload)
            elif msg.type == MsgType.Error:
                raise RuntimeError(f"TTS conversion failed: {msg}")

        if not audio_data:
            raise RuntimeError("No audio data received")
```

```python
        with open("unidirectional_stream.mp3", "wb") as f:
            f.write(audio_data)
        logger.info(f"Audio received: {len(audio_data)}, saved to unidirectional_stream.mp3")
    finally:
        await websocket.close()
        logger.info("Connection closed")

if __name__ == "__main__":
    asyncio.run(main())
```

HTTP 接口

```python
import requests
import json
import base64
import os

url = "https://openspeech.bytedance.com/api/v3/plan/tts/unidirectional"

def tts_http_stream():
    headers = {
        "X-Api-Key": "your_api_key",
        "X-Api-Resource-Id": "seed-tts-2.0",
        "Content-Type": "application/json",
        "Connection": "keep-alive",
        "X-Control-Require-Usage-Tokens-Return": "*"
    }

    payload = {
        "req_params": {
            "text": "你好，这是通过 HTTP 接口合成的语音。",
            "speaker": "zh_female_vv_uranus_bigtts",
            "audio_params": {
                "format": "mp3",
                "sample_rate": 24000,
            }
        }
    }
    session = requests.Session()
    response = None
    try:
        response = session.post(url, headers=headers, json=payload, stream=True)
```

```Python
        audio_data = bytearray()
        total_audio_size = 0
        for chunk in response.iter_lines(decode_unicode=True):
            if not chunk:
                continue
            data = json.loads(chunk)
            print(f"json data:{data}")
            if data.get("code", 0) == 0 and "data" in data and data["data"]:
                chunk_audio = base64.b64decode(data["data"])
                audio_size = len(chunk_audio)
                total_audio_size += audio_size
                audio_data.extend(chunk_audio)
            if data.get("code", 0) == 20000000:
                break
            if data.get("code", 0) > 0:
                print(f"error response:{data}")
                break

        if audio_data:
            if not os.path.exists("tts"):
                os.makedirs("tts")
            output_file = os.path.join("tts/", f"tts_test.mp3")
            with open(output_file, "wb") as f:
                f.write(audio_data)
            print(f"file size: {len(audio_data) / 1024:.2f} KB")

            os.chmod(output_file, 0o644)

    except Exception as e:
        print(f"request error: {e}")
    finally:
        if response:
            response.close()
        session.close()

if __name__ == "__main__":
    tts_http_stream()
```

流式语音识别（ASR Streaming）

流式语音识别支持通过 WebSocket 将实时语音流转写为文本，适用于实时转写、语音助手等场景。

接口信息

根据推荐场景选择对应的模式，对应的参数及交互流程可参见对应的 API 文档。

版权所有©北京火山引擎科技有限公司

接口类型	接口协议	接口地址	推荐场景	API 文档
双流接口	WebSocket	`wss://openspeech.bytedance.com/api/v3/plan/sauc/bigmodel_async`	边发送音频边实时返回识别结果，适合实时转写、语音助手等低延迟场景。	大模型流式语音识别 API
单流接口	WebSocket	`wss://openspeech.bytedance.com/api/v3/plan/sauc/bigmodel_nostream`	流式发送音频，待全部发送完成或超过 15s 后统一返回高精度结果，适合准确率优先的场景。	大模型流式语音识别 API

代码示例

> **注意**
> 代码示例中 `X-Api-Key` 需要设置为 专属 API Key。

双流接口

```python
import asyncio
import aiohttp
import json
import struct
import gzip
import uuid
import logging
import os
import subprocess
from typing import Optional, List, Dict, Any, Tuple, AsyncGenerator

# 配置日志
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('run.log'),
        logging.StreamHandler()
    ]
)
logger = logging.getLogger(__name__)
```

```python
# 常量定义
DEFAULT_SAMPLE_RATE = 16000

class ProtocolVersion:
    V1 = 0b0001

class MessageType:
    CLIENT_FULL_REQUEST = 0b0001
    CLIENT_AUDIO_ONLY_REQUEST = 0b0010
    SERVER_FULL_RESPONSE = 0b1001
    SERVER_ERROR_RESPONSE = 0b1111

class MessageTypeSpecificFlags:
    NO_SEQUENCE = 0b0000
    POS_SEQUENCE = 0b0001
    NEG_SEQUENCE = 0b0010
    NEG_WITH_SEQUENCE = 0b0011

class SerializationType:
    NO_SERIALIZATION = 0b0000
    JSON = 0b0001

class CompressionType:
    GZIP = 0b0001

class Config:
    def __init__(self):
        # 填入新版控制台获取的 API Key 和 Resource ID
        self.api_key = "your_api_key"
        self.resource_id = "volc.seedasr.sauc.duration"

config = Config()

class CommonUtils:
    @staticmethod
    def gzip_compress(data: bytes) -> bytes:
        return gzip.compress(data)

    @staticmethod
    def gzip_decompress(data: bytes) -> bytes:
        return gzip.decompress(data)

    @staticmethod
    def judge_wav(data: bytes) -> bool:
        if len(data) < 44:
            return False
        return data[:4] == b'RIFF' and data[8:12] == b'WAVE'
```

```
    @staticmethod
    def convert_wav_with_path(audio_path: str, sample_rate: int = DEFAULT_SAMPLE_RATE) -> bytes:
        try:
            cmd = [
                "ffmpeg", "-v", "quiet", "-y", "-i", audio_path,
                "-acodec", "pcm_s16le", "-ac", "1", "-ar", str(sample_rate),
                "-f", "wav", "-"
            ]
            result = subprocess.run(cmd, check=True, stdout=subprocess.PIPE, stderr=subprocess.PIPE)

            return result.stdout
        except subprocess.CalledProcessError as e:
            logger.error(f"FFmpeg conversion failed: {e.stderr.decode()}")
            raise RuntimeError(f"Audio conversion failed: {e.stderr.decode()}")

    @staticmethod
    def read_wav_info(data: bytes) -> Tuple[int, int, int, int, bytes]:
        if len(data) < 44:
            raise ValueError("Invalid WAV file: too short")

        # 解析WAV头
        chunk_id = data[:4]
        if chunk_id != b'RIFF':
            raise ValueError("Invalid WAV file: not RIFF format")

        format_ = data[8:12]
        if format_ != b'WAVE':
            raise ValueError("Invalid WAV file: not WAVE format")

        # 解析fmt子块
        audio_format = struct.unpack('<H', data[20:22])[0]
        num_channels = struct.unpack('<H', data[22:24])[0]
        sample_rate = struct.unpack('<I', data[24:28])[0]
        bits_per_sample = struct.unpack('<H', data[34:36])[0]

        # 查找data子块
        pos = 36
        while pos < len(data) - 8:
            subchunk_id = data[pos:pos+4]
            subchunk_size = struct.unpack('<I', data[pos+4:pos+8])[0]
            if subchunk_id == b'data':
                wave_data = data[pos+8:pos+8+subchunk_size]
                return (
                    num_channels,
                    bits_per_sample // 8,
                    sample_rate,
```

```
                subchunk_size // (num_channels * (bits_per_sample // 8)),
                wave_data
            )
        pos += 8 + subchunk_size

    raise ValueError("Invalid WAV file: no data subchunk found")

class AsrRequestHeader:
    def __init__(self):
        self.message_type = MessageType.CLIENT_FULL_REQUEST
        self.message_type_specific_flags = MessageTypeSpecificFlags.POS_SEQUENCE
        self.serialization_type = SerializationType.JSON
        self.compression_type = CompressionType.GZIP
        self.reserved_data = bytes([0x00])

    def with_message_type(self, message_type: int) -> 'AsrRequestHeader':
        self.message_type = message_type
        return self

    def with_message_type_specific_flags(self, flags: int) -> 'AsrRequestHeader':
        self.message_type_specific_flags = flags
        return self

    def with_serialization_type(self, serialization_type: int) -> 'AsrRequestHeader':
        self.serialization_type = serialization_type
        return self

    def with_compression_type(self, compression_type: int) -> 'AsrRequestHeader':
        self.compression_type = compression_type
        return self

    def with_reserved_data(self, reserved_data: bytes) -> 'AsrRequestHeader':
        self.reserved_data = reserved_data
        return self

    def to_bytes(self) -> bytes:
        header = bytearray()
        header.append((ProtocolVersion.V1 << 4) | 1)
        header.append((self.message_type << 4) | self.message_type_specific_flags)
        header.append((self.serialization_type << 4) | self.compression_type)
        header.extend(self.reserved_data)
        return bytes(header)

    @staticmethod
    def default_header() -> 'AsrRequestHeader':
        return AsrRequestHeader()
```

```
class RequestBuilder:
    @staticmethod
    def new_auth_headers() -> Dict[str, str]:
        reqid = str(uuid.uuid4())
        return {
            "X-Api-Key": config.api_key,
            "X-Api-Resource-Id": config.resource_id,
            "X-Api-Request-Id": reqid,
            "X-Api-Connect-Id": reqid,
            "X-Api-Sequence": "-1",
        }

    @staticmethod
    def new_full_client_request(seq: int) -> bytes:  # 添加seq参数
        header = AsrRequestHeader.default_header() \
            .with_message_type_specific_flags(MessageTypeSpecificFlags.POS_SEQUENCE)

        payload = {
            "user": {
                "uid": "demo_uid"
            },
            "audio": {
                "format": "wav",
                "codec": "raw",
                "rate": 16000,
                "bits": 16,
                "channel": 1
            },
            "request": {
                "model_name": "bigmodel",
                "enable_itn": True,
                "enable_punc": True,
                "enable_ddc": True,
                "show_utterances": True,
                "enable_nonstream": False
            }
        }

        payload_bytes = json.dumps(payload).encode('utf-8')
        compressed_payload = CommonUtils.gzip_compress(payload_bytes)
        payload_size = len(compressed_payload)

        request = bytearray()
        request.extend(header.to_bytes())
        request.extend(struct.pack('>i', seq))  # 使用传入的seq
        request.extend(struct.pack('>I', payload_size))
        request.extend(compressed_payload)
```

```
        return bytes(request)

    @staticmethod
    def new_audio_only_request(seq: int, segment: bytes, is_last: bool = False) -> bytes:
        header = AsrRequestHeader.default_header()
        if is_last:  # 最后一个包特殊处理
            header.with_message_type_specific_flags(MessageTypeSpecificFlags.NEG_WITH_SEQUENCE)
            seq = -seq  # 设为负值
        else:
            header.with_message_type_specific_flags(MessageTypeSpecificFlags.POS_SEQUENCE)
        header.with_message_type(MessageType.CLIENT_AUDIO_ONLY_REQUEST)

        request = bytearray()
        request.extend(header.to_bytes())
        request.extend(struct.pack('>i', seq))

        compressed_segment = CommonUtils.gzip_compress(segment)
        request.extend(struct.pack('>I', len(compressed_segment)))
        request.extend(compressed_segment)

        return bytes(request)

class AsrResponse:
    def __init__(self):
        self.code = 0
        self.event = 0
        self.is_last_package = False
        self.payload_sequence = 0
        self.payload_size = 0
        self.payload_msg = None

    def to_dict(self) -> Dict[str, Any]:
        return {
            "code": self.code,
            "event": self.event,
            "is_last_package": self.is_last_package,
            "payload_sequence": self.payload_sequence,
            "payload_size": self.payload_size,
            "payload_msg": self.payload_msg
        }

class ResponseParser:
    @staticmethod
    def parse_response(msg: bytes) -> AsrResponse:
        response = AsrResponse()
```

```
        header_size = msg[0] & 0x0f
        message_type = msg[1] >> 4
        message_type_specific_flags = msg[1] & 0x0f
        serialization_method = msg[2] >> 4
        message_compression = msg[2] & 0x0f

        payload = msg[header_size*4:]

        # 解析message_type_specific_flags
        if message_type_specific_flags & 0x01:
            response.payload_sequence = struct.unpack('>i', payload[:4])[0]
            payload = payload[4:]
        if message_type_specific_flags & 0x02:
            response.is_last_package = True
        if message_type_specific_flags & 0x04:
            response.event = struct.unpack('>i', payload[:4])[0]
            payload = payload[4:]

        # 解析message_type
        if message_type == MessageType.SERVER_FULL_RESPONSE:
            response.payload_size = struct.unpack('>I', payload[:4])[0]
            payload = payload[4:]
        elif message_type == MessageType.SERVER_ERROR_RESPONSE:
            response.code = struct.unpack('>i', payload[:4])[0]
            response.payload_size = struct.unpack('>I', payload[4:8])[0]
            payload = payload[8:]

        if not payload:
            return response

        # 解压缩
        if message_compression == CompressionType.GZIP:
            try:
                payload = CommonUtils.gzip_decompress(payload)
            except Exception as e:
                logger.error(f"Failed to decompress payload: {e}")
                return response

        # 解析payload
        try:
            if serialization_method == SerializationType.JSON:
                response.payload_msg = json.loads(payload.decode('utf-8'))
        except Exception as e:
            logger.error(f"Failed to parse payload: {e}")

        return response
```

```
class AsrWsClient:
    def __init__(self, url: str, segment_duration: int = 200):
        self.seq = 1
        self.url = url
        self.segment_duration = segment_duration
        self.conn = None
        self.session = None  # 添加session引用

    async def __aenter__(self):
        self.session = aiohttp.ClientSession()
        return self

    async def __aexit__(self, exc_type, exc, tb):
        if self.conn and not self.conn.closed:
            await self.conn.close()
        if self.session and not self.session.closed:
            await self.session.close()

    async def read_audio_data(self, file_path: str) -> bytes:
        try:
            with open(file_path, 'rb') as f:
                content = f.read()

            if not CommonUtils.judge_wav(content):
                logger.info("Converting audio to WAV format...")
                content = CommonUtils.convert_wav_with_path(file_path, DEFAULT_SAMPLE_RATE)

            return content
        except Exception as e:
            logger.error(f"Failed to read audio data: {e}")
            raise

    def get_segment_size(self, content: bytes) -> int:
        try:
            channel_num, samp_width, frame_rate, _, _ = CommonUtils.read_wav_info(content)[:5]
            size_per_sec = channel_num * samp_width * frame_rate
            segment_size = size_per_sec * self.segment_duration // 1000
            return segment_size
        except Exception as e:
            logger.error(f"Failed to calculate segment size: {e}")
            raise

    async def create_connection(self) -> None:
        headers = RequestBuilder.new_auth_headers()
        try:
            self.conn = await self.session.ws_connect(  # 使用self.session
                self.url,
```

```
            headers=headers
        )
        logger.info(f"Connected to {self.url}")
    except Exception as e:
        logger.error(f"Failed to connect to WebSocket: {e}")
        raise

async def send_full_client_request(self) -> None:
    request = RequestBuilder.new_full_client_request(self.seq)
    self.seq += 1  # 发送后递增
    try:
        await self.conn.send_bytes(request)
        logger.info(f"Sent full client request with seq: {self.seq-1}")

        msg = await self.conn.receive()
        if msg.type == aiohttp.WSMsgType.BINARY:
            response = ResponseParser.parse_response(msg.data)
            logger.info(f"Received response: {response.to_dict()}")
        else:
            logger.error(f"Unexpected message type: {msg.type}")
    except Exception as e:
        logger.error(f"Failed to send full client request: {e}")
        raise

async def send_messages(self, segment_size: int, content: bytes) -> AsyncGenerator[None, None]:
    audio_segments = self.split_audio(content, segment_size)
    total_segments = len(audio_segments)

    for i, segment in enumerate(audio_segments):
        is_last = (i == total_segments - 1)
        request = RequestBuilder.new_audio_only_request(
            self.seq,
            segment,
            is_last=is_last
        )
        await self.conn.send_bytes(request)
        logger.info(f"Sent audio segment with seq: {self.seq} (last: {is_last})")

        if not is_last:
            self.seq += 1

        await asyncio.sleep(self.segment_duration / 1000) # 逐个发送，间隔时间模拟实时流
        # 让出控制权，允许接收消息
        yield

async def recv_messages(self) -> AsyncGenerator[AsrResponse, None]:
    try:
```

```
            async for msg in self.conn:
                if msg.type == aiohttp.WSMsgType.BINARY:
                    response = ResponseParser.parse_response(msg.data)
                    yield response

                    if response.is_last_package or response.code != 0:
                        break
                elif msg.type == aiohttp.WSMsgType.ERROR:
                    logger.error(f"WebSocket error: {msg.data}")
                    break
                elif msg.type == aiohttp.WSMsgType.CLOSED:
                    logger.info("WebSocket connection closed")
                    break
        except Exception as e:
            logger.error(f"Error receiving messages: {e}")
            raise

    async def start_audio_stream(self, segment_size: int, content: bytes) ->
AsyncGenerator[AsrResponse, None]:
        async def sender():
            async for _ in self.send_messages(segment_size, content):
                pass

        # 启动发送和接收任务
        sender_task = asyncio.create_task(sender())

        try:
            async for response in self.recv_messages():
                yield response
        finally:
            sender_task.cancel()
            try:
                await sender_task
            except asyncio.CancelledError:
                pass

    @staticmethod
    def split_audio(data: bytes, segment_size: int) -> List[bytes]:
        if segment_size <= 0:
            return []

        segments = []
        for i in range(0, len(data), segment_size):
            end = i + segment_size
            if end > len(data):
                end = len(data)
            segments.append(data[i:end])
```

```
        return segments

    async def execute(self, file_path: str) -> AsyncGenerator[AsrResponse, None]:
        if not file_path:
            raise ValueError("File path is empty")

        if not self.url:
            raise ValueError("URL is empty")

        self.seq = 1

        try:
            # 1. 读取音频文件
            content = await self.read_audio_data(file_path)

            # 2. 计算分段大小
            segment_size = self.get_segment_size(content)

            # 3. 创建WebSocket连接
            await self.create_connection()

            # 4. 发送完整客户端请求
            await self.send_full_client_request()

            # 5. 启动音频流处理
            async for response in self.start_audio_stream(segment_size, content):
                yield response

        except Exception as e:
            logger.error(f"Error in ASR execution: {e}")
            raise
        finally:
            if self.conn:
                await self.conn.close()

async def main():
    # 配置信息
    audio_file_path = "your_file_path" # 替换为你的音频文件路径
    ws_url = "wss://openspeech.bytedance.com/api/v3/plan/sauc/bigmodel_async"
    seg_duration = 200 # 每次发送的音频片段时长(ms)

    async with AsrWsClient(ws_url, seg_duration) as client:
        try:
            async for response in client.execute(audio_file_path):
                logger.info(f"Received response: {json.dumps(response.to_dict(), indent=2,
ensure_ascii=False)}")
        except Exception as e:
```

```
Python    logger.error(f"ASR processing failed: {e}")

if __name__ == "__main__":
    asyncio.run(main())
```

单流接口

```
import asyncio
import aiohttp
import json
import struct
import gzip
import uuid
import logging
import os
import subprocess
from typing import Optional, List, Dict, Any, Tuple, AsyncGenerator

# 配置日志
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('run.log'),
        logging.StreamHandler()
    ]
)
logger = logging.getLogger(__name__)

# 常量定义
DEFAULT_SAMPLE_RATE = 16000

class ProtocolVersion:
    V1 = 0b0001

class MessageType:
    CLIENT_FULL_REQUEST = 0b0001
    CLIENT_AUDIO_ONLY_REQUEST = 0b0010
    SERVER_FULL_RESPONSE = 0b1001
    SERVER_ERROR_RESPONSE = 0b1111

class MessageTypeSpecificFlags:
    NO_SEQUENCE = 0b0000
    POS_SEQUENCE = 0b0001
```

```
    NEG_SEQUENCE = 0b0010
    NEG_WITH_SEQUENCE = 0b0011

class SerializationType:
    NO_SERIALIZATION = 0b0000
    JSON = 0b0001

class CompressionType:
    GZIP = 0b0001

class Config:
    def __init__(self):
        # 填入新版控制台获取的 API Key 和 Resource ID
        self.api_key = "your_api_key"
        self.resource_id = "volc.seedasr.sauc.duration"

config = Config()

class CommonUtils:
    @staticmethod
    def gzip_compress(data: bytes) -> bytes:
        return gzip.compress(data)

    @staticmethod
    def gzip_decompress(data: bytes) -> bytes:
        return gzip.decompress(data)

    @staticmethod
    def judge_wav(data: bytes) -> bool:
        if len(data) < 44:
            return False
        return data[:4] == b'RIFF' and data[8:12] == b'WAVE'

    @staticmethod
    def convert_wav_with_path(audio_path: str, sample_rate: int = DEFAULT_SAMPLE_RATE) -> bytes:
        try:
            cmd = [
                "ffmpeg", "-v", "quiet", "-y", "-i", audio_path,
                "-acodec", "pcm_s16le", "-ac", "1", "-ar", str(sample_rate),
                "-f", "wav", "-"
            ]
            result = subprocess.run(cmd, check=True, stdout=subprocess.PIPE, stderr=subprocess.PIPE)

            return result.stdout
        except subprocess.CalledProcessError as e:
            logger.error(f"FFmpeg conversion failed: {e.stderr.decode()}")
            raise RuntimeError(f"Audio conversion failed: {e.stderr.decode()}")
```

```
    @staticmethod
    def read_wav_info(data: bytes) -> Tuple[int, int, int, int, bytes]:
        if len(data) < 44:
            raise ValueError("Invalid WAV file: too short")

        # 解析WAV头
        chunk_id = data[:4]
        if chunk_id != b'RIFF':
            raise ValueError("Invalid WAV file: not RIFF format")

        format_ = data[8:12]
        if format_ != b'WAVE':
            raise ValueError("Invalid WAV file: not WAVE format")

        # 解析fmt子块
        audio_format = struct.unpack('<H', data[20:22])[0]
        num_channels = struct.unpack('<H', data[22:24])[0]
        sample_rate = struct.unpack('<I', data[24:28])[0]
        bits_per_sample = struct.unpack('<H', data[34:36])[0]

        # 查找data子块
        pos = 36
        while pos < len(data) - 8:
            subchunk_id = data[pos:pos+4]
            subchunk_size = struct.unpack('<I', data[pos+4:pos+8])[0]
            if subchunk_id == b'data':
                wave_data = data[pos+8:pos+8+subchunk_size]
                return (
                    num_channels,
                    bits_per_sample // 8,
                    sample_rate,
                    subchunk_size // (num_channels * (bits_per_sample // 8)),
                    wave_data
                )
            pos += 8 + subchunk_size

        raise ValueError("Invalid WAV file: no data subchunk found")

class AsrRequestHeader:
    def __init__(self):
        self.message_type = MessageType.CLIENT_FULL_REQUEST
        self.message_type_specific_flags = MessageTypeSpecificFlags.POS_SEQUENCE
        self.serialization_type = SerializationType.JSON
        self.compression_type = CompressionType.GZIP
        self.reserved_data = bytes([0x00])
```

```
    def with_message_type(self, message_type: int) -> 'AsrRequestHeader':
        self.message_type = message_type
        return self

    def with_message_type_specific_flags(self, flags: int) -> 'AsrRequestHeader':
        self.message_type_specific_flags = flags
        return self

    def with_serialization_type(self, serialization_type: int) -> 'AsrRequestHeader':
        self.serialization_type = serialization_type
        return self

    def with_compression_type(self, compression_type: int) -> 'AsrRequestHeader':
        self.compression_type = compression_type
        return self

    def with_reserved_data(self, reserved_data: bytes) -> 'AsrRequestHeader':
        self.reserved_data = reserved_data
        return self

    def to_bytes(self) -> bytes:
        header = bytearray()
        header.append((ProtocolVersion.V1 << 4) | 1)
        header.append((self.message_type << 4) | self.message_type_specific_flags)
        header.append((self.serialization_type << 4) | self.compression_type)
        header.extend(self.reserved_data)
        return bytes(header)

    @staticmethod
    def default_header() -> 'AsrRequestHeader':
        return AsrRequestHeader()

class RequestBuilder:
    @staticmethod
    def new_auth_headers() -> Dict[str, str]:
        reqid = str(uuid.uuid4())
        return {
            "X-Api-Key": config.api_key,
            "X-Api-Resource-Id": config.resource_id,
            "X-Api-Request-Id": reqid,
            "X-Api-Connect-Id": reqid,
            "X-Api-Sequence": "-1",
        }

    @staticmethod
    def new_full_client_request(seq: int) -> bytes:  # 添加seq参数
        header = AsrRequestHeader.default_header() \
```

```
            .with_message_type_specific_flags(MessageTypeSpecificFlags.POS_SEQUENCE)

        payload = {
            "user": {
                "uid": "demo_uid"
            },
            "audio": {
                "format": "wav",
                "codec": "raw",
                "rate": 16000,
                "bits": 16,
                "channel": 1
            },
            "request": {
                "model_name": "bigmodel",
                "enable_itn": True,
                "enable_punc": True,
                "enable_ddc": True,
                "show_utterances": True,
                "enable_nonstream": False
            }
        }

        payload_bytes = json.dumps(payload).encode('utf-8')
        compressed_payload = CommonUtils.gzip_compress(payload_bytes)
        payload_size = len(compressed_payload)

        request = bytearray()
        request.extend(header.to_bytes())
        request.extend(struct.pack('>i', seq))  # 使用传入的seq
        request.extend(struct.pack('>I', payload_size))
        request.extend(compressed_payload)

        return bytes(request)

    @staticmethod
    def new_audio_only_request(seq: int, segment: bytes, is_last: bool = False) -> bytes:
        header = AsrRequestHeader.default_header()
        if is_last:  # 最后一个包特殊处理
            header.with_message_type_specific_flags(MessageTypeSpecificFlags.NEG_WITH_SEQUENCE)
            seq = -seq  # 设为负值
        else:
            header.with_message_type_specific_flags(MessageTypeSpecificFlags.POS_SEQUENCE)
        header.with_message_type(MessageType.CLIENT_AUDIO_ONLY_REQUEST)

        request = bytearray()
        request.extend(header.to_bytes())
```

```
        request.extend(struct.pack('>i', seq))

        compressed_segment = CommonUtils.gzip_compress(segment)
        request.extend(struct.pack('>I', len(compressed_segment)))
        request.extend(compressed_segment)

        return bytes(request)

class AsrResponse:
    def __init__(self):
        self.code = 0
        self.event = 0
        self.is_last_package = False
        self.payload_sequence = 0
        self.payload_size = 0
        self.payload_msg = None

    def to_dict(self) -> Dict[str, Any]:
        return {
            "code": self.code,
            "event": self.event,
            "is_last_package": self.is_last_package,
            "payload_sequence": self.payload_sequence,
            "payload_size": self.payload_size,
            "payload_msg": self.payload_msg
        }

class ResponseParser:
    @staticmethod
    def parse_response(msg: bytes) -> AsrResponse:
        response = AsrResponse()

        header_size = msg[0] & 0x0f
        message_type = msg[1] >> 4
        message_type_specific_flags = msg[1] & 0x0f
        serialization_method = msg[2] >> 4
        message_compression = msg[2] & 0x0f

        payload = msg[header_size*4:]

        # 解析message_type_specific_flags
        if message_type_specific_flags & 0x01:
            response.payload_sequence = struct.unpack('>i', payload[:4])[0]
            payload = payload[4:]
        if message_type_specific_flags & 0x02:
            response.is_last_package = True
        if message_type_specific_flags & 0x04:
```

```
            response.event = struct.unpack('>i', payload[:4])[0]
            payload = payload[4:]

        # 解析message_type
        if message_type == MessageType.SERVER_FULL_RESPONSE:
            response.payload_size = struct.unpack('>I', payload[:4])[0]
            payload = payload[4:]
        elif message_type == MessageType.SERVER_ERROR_RESPONSE:
            response.code = struct.unpack('>i', payload[:4])[0]
            response.payload_size = struct.unpack('>I', payload[4:8])[0]
            payload = payload[8:]

        if not payload:
            return response

        # 解压缩
        if message_compression == CompressionType.GZIP:
            try:
                payload = CommonUtils.gzip_decompress(payload)
            except Exception as e:
                logger.error(f"Failed to decompress payload: {e}")
                return response

        # 解析payload
        try:
            if serialization_method == SerializationType.JSON:
                response.payload_msg = json.loads(payload.decode('utf-8'))
        except Exception as e:
            logger.error(f"Failed to parse payload: {e}")

        return response

class AsrWsClient:
    def __init__(self, url: str, segment_duration: int = 200):
        self.seq = 1
        self.url = url
        self.segment_duration = segment_duration
        self.conn = None
        self.session = None  # 添加session引用

    async def __aenter__(self):
        self.session = aiohttp.ClientSession()
        return self

    async def __aexit__(self, exc_type, exc, tb):
        if self.conn and not self.conn.closed:
            await self.conn.close()
```

```
        if self.session and not self.session.closed:
            await self.session.close()

    async def read_audio_data(self, file_path: str) -> bytes:
        try:
            with open(file_path, 'rb') as f:
                content = f.read()

            if not CommonUtils.judge_wav(content):
                logger.info("Converting audio to WAV format...")
                content = CommonUtils.convert_wav_with_path(file_path, DEFAULT_SAMPLE_RATE)

            return content
        except Exception as e:
            logger.error(f"Failed to read audio data: {e}")
            raise

    def get_segment_size(self, content: bytes) -> int:
        try:
            channel_num, samp_width, frame_rate, _, _ = CommonUtils.read_wav_info(content)[:5]
            size_per_sec = channel_num * samp_width * frame_rate
            segment_size = size_per_sec * self.segment_duration // 1000
            return segment_size
        except Exception as e:
            logger.error(f"Failed to calculate segment size: {e}")
            raise

    async def create_connection(self) -> None:
        headers = RequestBuilder.new_auth_headers()
        try:
            self.conn = await self.session.ws_connect(  # 使用self.session
                self.url,
                headers=headers
            )
            logger.info(f"Connected to {self.url}")
        except Exception as e:
            logger.error(f"Failed to connect to WebSocket: {e}")
            raise

    async def send_full_client_request(self) -> None:
        request = RequestBuilder.new_full_client_request(self.seq)
        self.seq += 1  # 发送后递增
        try:
            await self.conn.send_bytes(request)
            logger.info(f"Sent full client request with seq: {self.seq-1}")

            msg = await self.conn.receive()
```

```
            if msg.type == aiohttp.WSMsgType.BINARY:
                response = ResponseParser.parse_response(msg.data)
                logger.info(f"Received response: {response.to_dict()}")
            else:
                logger.error(f"Unexpected message type: {msg.type}")
        except Exception as e:
            logger.error(f"Failed to send full client request: {e}")
            raise

    async def send_messages(self, segment_size: int, content: bytes) -> AsyncGenerator[None, None]:
        audio_segments = self.split_audio(content, segment_size)
        total_segments = len(audio_segments)

        for i, segment in enumerate(audio_segments):
            is_last = (i == total_segments - 1)
            request = RequestBuilder.new_audio_only_request(
                self.seq,
                segment,
                is_last=is_last
            )
            await self.conn.send_bytes(request)
            logger.info(f"Sent audio segment with seq: {self.seq} (last: {is_last})")

            if not is_last:
                self.seq += 1

            await asyncio.sleep(self.segment_duration / 1000) # 逐个发送，间隔时间模拟实时流
            # 让出控制权，允许接收消息
            yield

    async def recv_messages(self) -> AsyncGenerator[AsrResponse, None]:
        try:
            async for msg in self.conn:
                if msg.type == aiohttp.WSMsgType.BINARY:
                    response = ResponseParser.parse_response(msg.data)
                    yield response

                    if response.is_last_package or response.code != 0:
                        break
                elif msg.type == aiohttp.WSMsgType.ERROR:
                    logger.error(f"WebSocket error: {msg.data}")
                    break
                elif msg.type == aiohttp.WSMsgType.CLOSED:
                    logger.info("WebSocket connection closed")
                    break
        except Exception as e:
            logger.error(f"Error receiving messages: {e}")
```

```
            raise

    async def start_audio_stream(self, segment_size: int, content: bytes) ->
AsyncGenerator[AsrResponse, None]:
        async def sender():
            async for _ in self.send_messages(segment_size, content):
                pass

        # 启动发送和接收任务
        sender_task = asyncio.create_task(sender())

        try:
            async for response in self.recv_messages():
                yield response
        finally:
            sender_task.cancel()
            try:
                await sender_task
            except asyncio.CancelledError:
                pass

    @staticmethod
    def split_audio(data: bytes, segment_size: int) -> List[bytes]:
        if segment_size <= 0:
            return []

        segments = []
        for i in range(0, len(data), segment_size):
            end = i + segment_size
            if end > len(data):
                end = len(data)
            segments.append(data[i:end])
        return segments

    async def execute(self, file_path: str) -> AsyncGenerator[AsrResponse, None]:
        if not file_path:
            raise ValueError("File path is empty")

        if not self.url:
            raise ValueError("URL is empty")

        self.seq = 1

        try:
            # 1. 读取音频文件
            content = await self.read_audio_data(file_path)
```

```python
            # 2. 计算分段大小
            segment_size = self.get_segment_size(content)

            # 3. 创建WebSocket连接
            await self.create_connection()

            # 4. 发送完整客户端请求
            await self.send_full_client_request()

            # 5. 启动音频流处理
            async for response in self.start_audio_stream(segment_size, content):
                yield response

        except Exception as e:
            logger.error(f"Error in ASR execution: {e}")
            raise
        finally:
            if self.conn:
                await self.conn.close()

async def main():
    # 配置信息
    audio_file_path = "your_file_path" # 替换为你的音频文件路径
    ws_url = "wss://openspeech.bytedance.com/api/v3/plan/sauc/bigmodel_nostream"
    seg_duration = 200 # 每次发送的音频片段时长(ms)

    async with AsrWsClient(ws_url, seg_duration) as client:
        try:
            async for response in client.execute(audio_file_path):
                logger.info(f"Received response: {json.dumps(response.to_dict(), indent=2,
ensure_ascii=False)}")
        except Exception as e:
            logger.error(f"ASR processing failed: {e}")

if __name__ == "__main__":
    asyncio.run(main())
```

接口接入建议

请求和响应参数：

- 始终记录 Response Header 中的 `X-Tt-Logid`，便于问题排查。
- 为每次连接/请求生成唯一的 UUID 作为 `X-Api-Connect-Id` 或 `X-Api-Request-Id`。

版权所有©北京火山引擎科技有限公司

计费说明

- **未开启超额后付费**：语音模型调用产生的费用使用套餐内 AFP 进行抵扣，不消耗其他资源包或账户余额。抵扣规则参见 套餐内 AFP 抵扣规则 。
- **开启超额后付费**：在套餐周期内额度未用尽时优先抵扣套餐额度；当套餐周期内额度用尽时，无需修改任何配置信息（如 Base URL、API Key、模型名称），系统将自动切换至超额后付费模式继续使用，并产生后付费账单。具体规则与操作参见 超额后付费规则 和 超额后付费管理 。

版权所有©北京火山引擎科技有限公司